

4_G1 GC 심화

- **Generational (세대별)**: 다른 컬렉터처럼 Heap을 Young / Old로 나눈다.
- **Incremental (점진적)**: Old 세대 공간 회수를 한 번에 하지 않고 여러 번에 걸쳐 조금씩 한다. 덕분에 pause가 짧아진다.
- **Parallel + Mostly Concurrent (병렬 + 대부분 동시)**: 일부 작업은 애플리케이션과 동시에, 일부는 여러 스레드로 병렬 처리한다.
- **Stop-The-World**: 실제 공간 회수는 STW pause 안에서 일어난다.
- **Evacuating (대피형)**: 공간 회수를 주로 '대피'로 한다. 살아있는 객체를 새 영역으로 복사하며 압축하고, 원래 공간은 재사용한다.

1. 왜 G1을 쓰며, 무엇을 목표로 하는가

G1의 목표: latency와 throughput 사이의 최상의 균형 유지.

G1이 겨냥하는 애플리케이션 환경은 다음과 같다.

- Heap 크기가 **수십 GB**
- Heap의 **50% 이상**이 라이브 데이터로 차 있음
- 객체 할당·승격 속도가 시간에 따라 크게 변동
- Heap에 **상당한 단편화**가 존재
- **수백 ms를 넘지 않는 예측 가능한 pause 목표**를 원함(긴 pause 회피)

G1은 작업의 일부를 애플리케이션과 동시에 수행한다. 이를 위해, 애플리케이션이 쓸 수 있었을 프로세서 자원을 **더 짧은 pause와 맞바꾼다**.

2. 작동 방식 — Region 기반 Heap

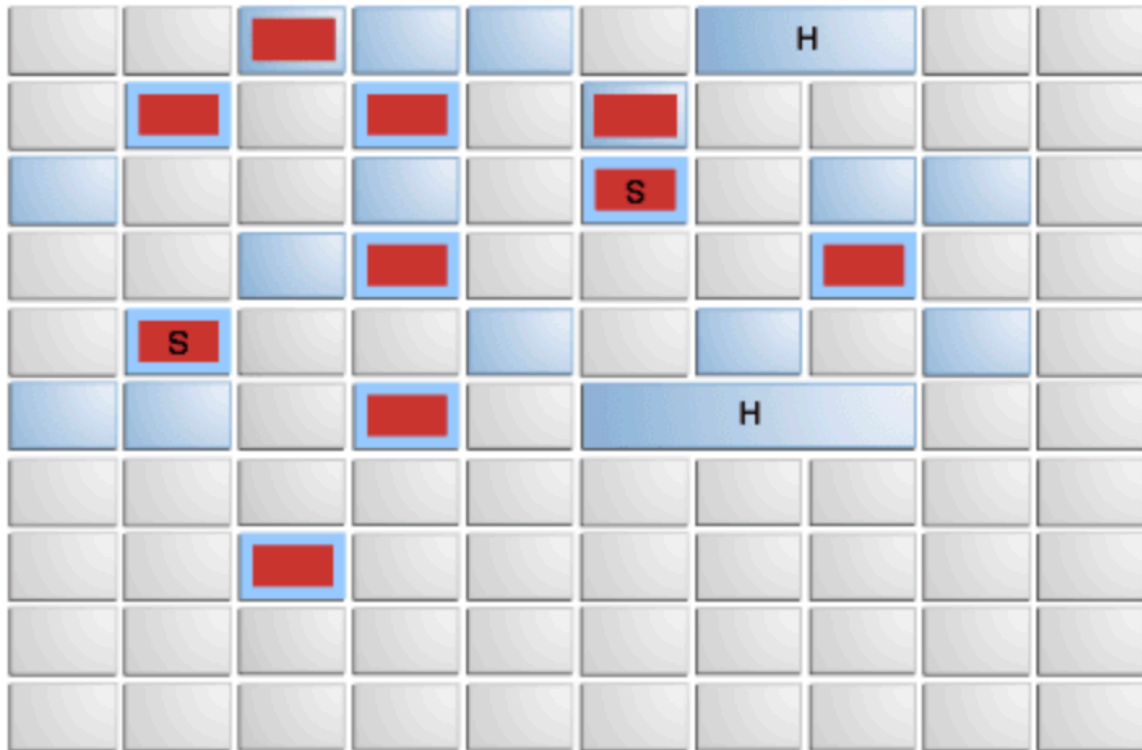
Region 기반 Heap 구조는 G1의 가장 큰 차별점이다.

Heap을 동일한 크기의 **region**들로 분할하고, region을 메모리 할당과 회수의 단위로 삼는다. 각 region은 시점에 따라 비어 있거나, Young(eden / survivor) 또는 Old로 동적으로 할당된다.

기존 컬렉터와의 결정적 차이는 이렇다.

- 세대가 메모리상 연속적일 필요가 없다.
- eden과 survivor가 여기선 흩어진 region들의 집합이다.

Figure 7-1 G1 Garbage Collector Heap Layout



Description of "Figure 7-1 G1 Garbage Collector Heap Layout "

기본값

- region 크기는 Heap 크기를 기준으로 약 2048개가 되도록 자동 계산된다.
- Young 세대는 Heap의 5~60% 사이에서, pause 목표에 맞춰 적응적으로 크기가 변한다.

G1은 이전 동작을 관찰해 비용 모델을 만들고, **쓰레기가 가장 많은(= 회수 효율이 가장 높은) 영역부터 회수한다**. 그래서 이름이 **Garbage-First**다. 즉, pause에 할당된 시간 안에서 가장 이득이 큰 region부터 처리한다.

G1은 두 phase를 번갈아 도는데, **Young-Only**와 **Space-Reclamation**이다.

2.1 Young-Only Phase

일반적인 young collection이 반복되며, 객체들을 Old로 승격시켜 Old를 점점 채운다.

Old 점유율이 임계치에 도달하면 전환이 시작된다.

- **Concurrent Start**: young collection을 하면서, 동시에 마킹을 시작한다.
- **Remark**: 마킹을 마무리하고, 전역 참조 처리·클래스 언로딩을 수행하며, 완전히 빈 region을 회수한다.
- **Cleanup**: Space-Reclamation 단계로 넘어갈지 결정한다.

2.2 Space-Reclamation Phase

- 여러 번의 **Mixed Collection**으로 진행된다.

- Young region과 일부 Old region을 함께 **evacuate**한다.
- Old region을 더 회수해봐야 이득이 적어지면 이 단계를 끝내고, 다시 Young-Only로 돌아간다.

2.3 Marking — SATB

마킹은 **SATB(Snapshot-At-The-Beginning)** 알고리즘을 사용한다.

마킹 시작 시점에 Heap의 가상 스냅샷을 찍어, 그 시점에 살아있던 객체는 마킹이 끝날 때까지 계속 살아 있는 것으로 간주한다.

부작용

- 마킹 도중에 죽은 객체도 이번 사이클에는 살아있는 것으로 취급되어, 불필요하게 유지될 수 있다 (floating garbage).

2.4 Remembered Set

G1이 Old region 하나를 개별적으로 evacuate하려면, 다른 region에서 이 region으로 향하는 참조를 알아야 한다. 이 **교차 참조 집합**이 **Remembered Set(RSet)** 이다.

RSet 유지·관리는 대부분 concurrent하게 이뤄진다. 성능을 위해 즉시 갱신하지 않고, 요청을 모아 배치로 처리한다.